

Interactive Local Text Document Search Engine: Comparing Classical, Semantic, Hybrid, and LLM- Based Retrieval Methods

Apurv Chokshi
Jarvis College of Computing and Digital Media
DePaul University
Chicago, USA
achoksh2@depaul.edu

Abstract— Large document collections are common in real-world settings such as universities, companies, research groups, and digital archives. Although document search systems already exist, the performance of a search system depends heavily on the retrieval method used and the type of query submitted by the user. This project develops an interactive local text document search engine and uses it as an experimental testbed to compare multiple information retrieval methods. The system indexes more than 2,000 documents from the 20 Newsgroups dataset and supports TF-IDF retrieval, semantic search using sentence embeddings, hybrid retrieval, and LLM-based query expansion. Results are evaluated using category-based relevance judgments and metrics such as Precision@K, Recall@K, F1-score, and Average Precision. Preliminary findings show that TF-IDF performs well for exact keyword queries, semantic search performs better for meaning-based queries, hybrid search provides a tunable balance, and LLM query expansion can be useful but may cause topic drift when not carefully controlled.

Keywords—*Information Retrieval; Local Search Engine; TF-IDF; Semantic Search; Hybrid Retrieval; Query Expansion*

I. INTRODUCTION

Large text collections are common in many real-world environments, including universities, companies, research labs, news archives, legal offices, and customer support systems. These collections may contain thousands of documents such as articles, reports, notes, emails, abstracts, or technical records. When the number of documents becomes large, manually opening and searching each file is inefficient. Information retrieval systems address this problem by automatically indexing documents, processing user queries, and ranking documents based on estimated relevance.

Search systems and digital libraries already exist, so the goal of this project is not to invent a completely new type of search engine. Instead, this project uses a local document search application as a controlled test environment for comparing different retrieval methods. The main question addressed in this work is: how do classical, semantic, hybrid, and LLM-based retrieval methods perform when applied to the same local text document search task?

The project implements an interactive local text document search engine using the 20 Newsgroups dataset. The system indexes more than 2,000 text documents and allows a user to enter queries and view ranked results. Each result includes a document ID, similarity score, document category, and a short snippet. The system supports multiple retrieval methods: TF-IDF with cosine similarity, semantic search using sentence embeddings, hybrid retrieval, and LLM-based query expansion.

TF-IDF retrieval represents the classical vector space model approach. It performs well when the words in the query directly overlap with the words in relevant documents. Semantic search uses dense sentence embeddings to compare documents and queries based on meaning rather than only exact word overlap. Hybrid retrieval combines sparse keyword-based retrieval and semantic retrieval using a tunable weighting parameter. LLM-based query expansion attempts to improve sparse retrieval by adding related terms to the original query, although this approach can introduce noise or topic drift if the generated expansion is not controlled carefully.

The system is evaluated using category-based relevance judgments from the 20 Newsgroups dataset. For example, if the query is related to space, documents from the sci.space category are treated as relevant. Retrieval quality is measured using Precision@K, Recall@K, F1-score, and Average Precision. These metrics make it possible to compare how well each method retrieves relevant documents and how effectively relevant results are ranked near the top.

The preliminary results show that no single retrieval method performs best for every query. TF-IDF is useful for exact keyword and entity-based searches, while semantic search is stronger for meaning-based queries such as definitions, phrases, or broader concepts. Hybrid retrieval provides a balance between both approaches, but its performance depends on the selected weighting parameter. LLM query expansion is an interesting advanced method, but in this project it produced mixed results because generated terms sometimes shifted the query away from the original topic. Overall, this project demonstrates that retrieval effectiveness depends on the query type, document collection, and retrieval method selected.

II. BACKGROUND

Information retrieval (IR) focuses on finding relevant information from a collection of documents in response to a user query. A basic IR system usually includes document preprocessing, indexing, query processing, matching, ranking, and evaluation. Traditional search systems often represent documents as bags of words, where the order of words is mostly ignored and documents are represented using term-based features. This approach is simple, efficient, and interpretable, which is why sparse retrieval methods are still widely used in many search applications.

One of the most common classical approaches is the vector space model. In this model, documents and queries are represented as vectors over a vocabulary of terms. Term weights are often calculated using TF-IDF, where term frequency measures how often a term appears in a document, and inverse document frequency reduces the importance of terms that appear in many documents. After creating document and query vectors, similarity can be measured using cosine similarity. This method is useful for keyword-based retrieval because it gives higher scores to documents that share important terms with the query [1].

However, sparse keyword-based retrieval has an important limitation: query-document vocabulary mismatch. A relevant document may discuss the same idea as the query but use different words. For example, a user may search for “space mission,” while a relevant document may use terms such as

“NASA launch,” “orbit,” or “lunar exploration.” In this case, TF-IDF may miss relevant documents because it depends heavily on exact term overlap. This limitation motivates the use of semantic retrieval methods.

Semantic retrieval uses dense vector representations, often produced by neural embedding models. Instead of representing documents only by individual word counts, embedding models represent the overall meaning of a query or document in a dense vector space. Sentence embedding models such as Sentence-BERT and MiniLM have been used to compare texts based on semantic similarity rather than exact keyword matching [2]. This makes semantic search useful when users describe an idea, definition, phrase, or concept using words that may not exactly match the document.

Hybrid retrieval combines sparse and dense retrieval methods. Sparse methods such as TF-IDF or BM25 are strong for exact keyword matching, while dense semantic methods are strong for meaning-based matching. A hybrid approach attempts to use both strengths by combining the scores from sparse and semantic retrieval. In this project, the hybrid score is controlled using an alpha parameter. A higher alpha gives more weight to TF-IDF, while a lower alpha gives more weight to semantic similarity. Hybrid retrieval is useful because many real queries contain both exact keywords and broader semantic intent.

Query expansion is another technique used to address vocabulary mismatch. Traditional query expansion adds related words, synonyms, or terms from pseudo-relevant documents to the original query. Recent work has explored using large language models for query rewriting and query expansion. For example, LLMs can generate paraphrased queries or create pseudo-documents that add related terms to the original query. Ayoub, Su, and Li studied LLM-based query rewriting and generative query expansion for sparse retrieval and found that LLM-generated expansions can improve sparse retrieval in several benchmark datasets [3]. However, they also observed that the effectiveness of LLM-based expansion can depend on the dataset and domain.

LLM query expansion can also introduce a problem called topic drift. If the generated expansion includes too many additional terms, the expanded query may move away from the original user intent. This is especially risky in sparse retrieval because the ranking model depends directly on the words in the query. If the expansion adds noisy or unrelated terms, the system may retrieve documents that match the expansion rather than the original query. For this reason, LLM-based expansion needs to be controlled carefully, especially when documents are short or contain limited context.

The present project builds on these ideas by implementing and comparing four retrieval methods in the same local

document search application: TF-IDF retrieval, semantic search, hybrid retrieval, and LLM-based query expansion. Rather than treating one method as universally best, the goal is to study how each method behaves under the same dataset, interface, and evaluation setup. This makes it possible to analyze which method works better for exact keyword searches, meaning-based searches, and cases where query expansion may help or hurt retrieval quality.

III. METHODOLOGY

The methodology of this project follows a standard information retrieval pipeline. The system first loads a document collection, preprocesses the text, builds an index, processes user queries, ranks matching documents, and then evaluates the ranked results. The overall goal is to use the same dataset and interface to compare multiple retrieval strategies under the same conditions.

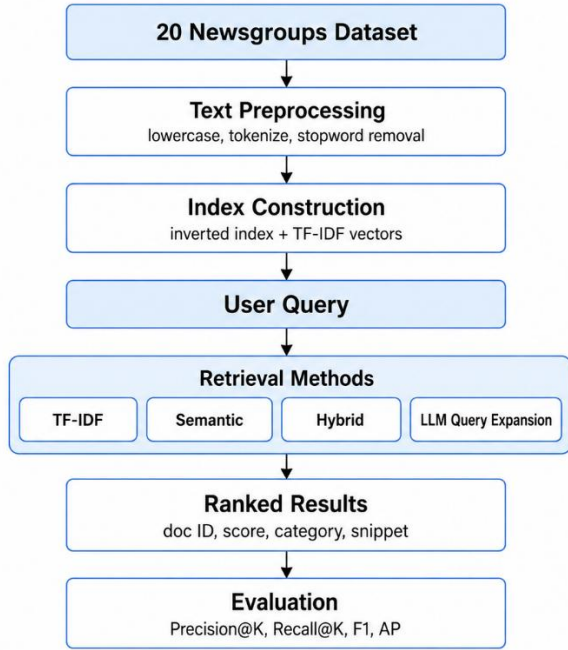


Figure 1. Overall architecture of the interactive local document search system.

A. System Pipeline

The first step is data loading. The system uses the 20 Newsgroups dataset, which contains documents from multiple topic categories. Each document is treated as an item in the local search collection. The dataset also provides category

labels, which are later used as relevance judgments during evaluation.

The second step is preprocessing. Each document is converted to lowercase, tokenized into words, and cleaned by removing stopwords and short or non-informative terms. This step reduces noise in the document representation and makes the indexing process more consistent. The same preprocessing function is also applied to user queries so that documents and queries are represented in a comparable way.

The third step is index construction. The system builds an inverted index, where each term points to the documents in which it appears. Each posting stores the document ID and the term frequency. This makes retrieval efficient because the system can find candidate documents by looking up query terms in the index instead of scanning all documents manually.

The fourth step is document representation and ranking. For the sparse retrieval baseline, documents and queries are represented as TF-IDF vectors. The system computes inverse document frequency values from the collection and then creates normalized document vectors. When a query is submitted, it is converted into a TF-IDF query vector and compared with candidate document vectors using cosine similarity. Documents are ranked in descending order of similarity score.

The fifth step is evaluation. The system evaluates ranked results using the category labels from the dataset. For a given query, a selected category is treated as the relevant class. Documents from that category are counted as relevant, while documents from other categories are counted as non-relevant. This allows the system to calculate Precision@K, Recall@K, F1-score, and Average Precision.

B. TF-IDF Retrieval

The first retrieval method is TF-IDF with cosine similarity. This method follows the classical vector space model. Each document is represented as a weighted vector of terms. Terms that occur often in a document receive higher term frequency values, while terms that appear across many documents receive lower inverse document frequency weights.

This method is useful for exact keyword matching. If a query contains specific terms that also appear in relevant documents, TF-IDF can rank those documents highly. However, it depends heavily on lexical overlap, so it may miss relevant documents that use different words to express the same idea.

C. Semantic Search

The second method is semantic search using sentence embeddings. In this method, documents and queries are converted into dense vector representations using a sentence-transformer model. Instead of comparing documents and queries based only on shared words, the system compares their meaning using cosine similarity between embedding vectors.

Semantic search is useful when the query and document are related in meaning but do not use the exact same vocabulary. For example, a query about “space travel” may retrieve documents about “NASA missions” or “orbital flight” even if the exact query words are not repeated.

D. Hybrid Retrieval

The third method is hybrid retrieval. This method combines the TF-IDF score and the semantic similarity score into a single final score. The hybrid score is controlled by an alpha parameter:

$$\text{Score}_{\text{hybrid}} = \alpha \cdot \text{Score}_{\text{TF-IDF}} + (1 - \alpha) \cdot \text{Score}_{\text{semantic}}$$

When alpha is close to 1, the system gives more importance to TF-IDF. When alpha is close to 0, it gives more importance to semantic similarity. This allows hybrid retrieval to balance exact keyword matching with meaning-based retrieval. In the experiments, the hybrid method is compared using the same queries as the other methods.

E. LLM-Based Query Expansion

The fourth method is LLM-based query expansion. This method is inspired by recent work on using large language models to improve sparse retrieval through query rewriting and query expansion. The goal is to reduce query-document mismatch by adding related terms to the original query before running sparse retrieval.

In this project, the LLM expansion method generates additional related terms for the user query. The expanded query is then passed to the TF-IDF retrieval system. To reduce the risk of the generated terms overpowering the original query, the original query is repeated before adding the expansion. This gives more weight to the user’s original intent.

This method is experimental because LLM-generated terms can sometimes improve retrieval by adding useful vocabulary, but they can also introduce unrelated terms. When this happens, the expanded query may shift away from the original topic, which is known as topic drift.

F. Method Comparison

All methods are compared using the same document collection, same queries, and same relevance labels. For each test query, the system returns the top K ranked documents for TF-IDF, semantic search, hybrid search, and LLM query expansion. The system then measures how many of the retrieved documents belong to the selected relevant category.

This comparison is designed to answer the main research question of the project: which retrieval method works better for different types of local document search queries? TF-IDF is expected to perform well for exact keyword queries, semantic search is expected to perform well for meaning-based queries, hybrid search is expected to provide a balanced result, and LLM query expansion is tested as an advanced method for improving sparse retrieval.

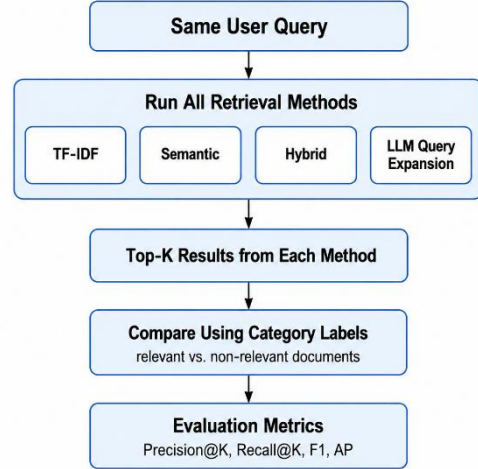


Figure 2. Method comparison process using the same query and same document collection.

IV. RESULTS

The system was evaluated using the 20 Newsgroups dataset. This dataset contains text documents from multiple topic categories such as science, sports, computers, vehicles, politics,

and religion. For this project, more than 2,000 documents were indexed and used for retrieval experiments. Each document has an associated category label, which makes the dataset useful for evaluating search results.

The main goal of the experiments was to compare how different retrieval methods perform on the same local document search task. The four methods compared were TF-IDF retrieval, semantic search, hybrid retrieval, and LLM-based query expansion. Each method was tested using the same user query and the same document collection. The system returned the top-K ranked documents for each method.

For evaluation, the category labels were used as relevance judgments. For example, if the query was related to space, the relevant category was selected as sci.space. Documents from that category were counted as relevant, while documents from other categories were counted as non-relevant. This approach allowed the system to calculate quantitative retrieval metrics without requiring manual relevance labeling.

The metrics used were Precision@K, Recall@K, F1-score, and Average Precision. Precision@K measures how many of the top-K retrieved documents are relevant. Recall@K measures how many relevant documents were found out of all relevant documents in the selected category. F1-score provides a balance between precision and recall. Average Precision also considers the ranking order by giving higher value when relevant documents appear closer to the top.

Figure 3 shows one example of the method comparison output from the Gradio interface. In this experiment, TF-IDF retrieved 8 relevant documents in the top 10 results, while semantic search and hybrid search both retrieved all 10 relevant documents. LLM query expansion also retrieved 8 relevant documents. Semantic and hybrid retrieval achieved the highest Precision@10, Recall@10, and F1-score for this query, showing that meaning-based matching and combined retrieval were more effective than keyword-only retrieval in this case.

Figure 4 shows a second query where the methods produced a different performance pattern. In this experiment, TF-IDF retrieved 5 relevant documents in the top 10 results, semantic search retrieved 7, hybrid search retrieved 6, and LLM query expansion retrieved 5. Semantic search had the highest Precision@10 and F1-score, while hybrid search achieved the highest Average Precision. These results show that retrieval performance changes depending on the query, and that different methods may be stronger under different query conditions.

Overall, I have done a bunch of experiments but I am showing only two examples. the experiments show that no single method is always best. TF-IDF performs well when the query contains exact keywords that appear in the documents. Semantic search often performs better when the query is meaning-based or when relevant documents use different vocabulary. Hybrid search can provide a balanced result by combining keyword matching and semantic similarity. LLM query expansion gave mixed results because the generated terms sometimes helped but could also introduce noise or cause topic drift.

Qualitatively, good results occurred when the query matched a clear topic category and the returned snippets were closely related to the user’s intent. Bad results occurred when a method retrieved documents from nearby but incorrect categories, or when LLM expansion added terms that shifted the query away from the original topic. These examples suggest that retrieval performance depends not only on the algorithm, but also on the query wording and the content density of the documents.

Method	Precision	Recall	F1	Avg. Precision	Relevant found
TF-IDF	0.8	0.0329	0.0632	1	8
Semantic	1	0.0412	0.0791	1	10
Hybrid	1	0.0412	0.0791	1	10
LLM Query Expansion	0.8	0.0329	0.0632	1	8

Figure 3. Method comparison results for Query A, showing semantic and hybrid retrieval outperforming TF-IDF and LLM query expansion.

Method	Precision	Recall	F1	Avg. Precision	Relevant found
TF-IDF	0.5	0.0181	0.0035	0.6593	5
Semantic	0.7	0.0254	0.0049	0.826	7
Hybrid	0.6	0.0217	0.0042	0.8722	6
LLM Query Expansion	0.5	0.0181	0.0035	0.8583	5

Figure 4. Method comparison results for Query B, showing different performance patterns across retrieval methods and illustrating query-level variation.

V. DISCUSSION

The experimental results show that retrieval performance depends strongly on the type of query being used. No single method performed best in every situation. Instead, each retrieval method showed different strengths and weaknesses depending on whether the query required exact keyword matching, semantic understanding, or a balance between both.

TF-IDF performed well when the query contained specific keywords that appeared directly in the documents. This makes TF-IDF useful for searches involving names, entities, technical terms, or exact phrases. Its main advantage is that it is simple, efficient, and interpretable. However, TF-IDF depends heavily on lexical overlap. If a relevant document uses different words from the query, TF-IDF may not rank it highly even if the document is topically relevant.

Semantic search performed better for queries where meaning was more important than exact word overlap. Because this method uses sentence embeddings, it can retrieve documents that are related in meaning even when the exact query words do not appear in the document. This was shown in the experiments where semantic search retrieved more relevant documents than TF-IDF for some queries. However, semantic search is less interpretable than TF-IDF because the ranking is based on dense vector similarity rather than visible term weights.

Hybrid retrieval provided a balance between keyword-based and meaning-based retrieval. In some experiments, hybrid search performed as well as semantic search, while in others it performed between TF-IDF and semantic search. The effectiveness of hybrid retrieval depends on the alpha value. A higher alpha gives more importance to TF-IDF, while a lower alpha gives more importance to semantic similarity. This means that alpha is an important hyperparameter, and different query types may require different alpha settings.

LLM-based query expansion produced mixed results. The goal of this method was to improve sparse retrieval by adding related terms to the original query. However, in this project, the method did not consistently improve over TF-IDF. One reason is that many documents in the selected dataset are short or have limited content. When the LLM generated extra terms, those terms sometimes introduced noise or shifted the query away from the original topic. This topic drift reduced the usefulness of the expanded query in some cases. The results suggest that LLM query expansion should be controlled carefully, especially for sparse retrieval systems.

One important observation from this project is that search quality depends not only on the retrieval algorithm but also on the user's query. If the user knows the exact terms used in the collection, TF-IDF can be effective. If the user searches using broader meaning or related concepts, semantic search may work better. If the query contains both exact terms and broader intent, hybrid retrieval may be useful. This shows why

modern search systems often combine multiple retrieval strategies instead of relying on only one method.

There are several opportunities for improvement. First, the system could test more alpha values for hybrid retrieval and select the best value automatically. Second, the LLM query expansion method could be improved by generating only a small number of controlled keywords instead of longer text expansions. Third, the system could use a larger or denser dataset where each document contains more detailed content. Finally, future work could add BM25, neural reranking, or user relevance feedback to further improve ranking quality.

VI. CONCLUSION

This project developed an interactive local text document search engine and used it to compare multiple retrieval methods on the same document collection. The system indexed more than 2,000 documents from the 20 Newsgroups dataset and supported TF-IDF retrieval, semantic search, hybrid retrieval, and LLM-based query expansion. A Gradio interface was also created to allow users to search documents, evaluate individual queries, and compare methods side by side.

The main finding is that retrieval effectiveness depends on the user's query and the type of information being searched. TF-IDF worked well for exact keyword searches, especially when the user was looking for specific names, entities, or terms already present in the documents. Semantic search performed better when the query was more meaning-based, such as when the user was looking for a definition, explanation, phrase, or broader idea. Hybrid search provided a useful balance between keyword matching and semantic matching, although its performance depended on the alpha parameter.

LLM-based query expansion was an interesting advanced method, but it did not consistently improve the results in this project. In some cases, the generated expansion added noise or shifted the query away from the original topic. This suggests that query expansion must be controlled carefully, especially when working with shorter documents or sparse text collections.

Overall, this project showed that building a search system is not only about returning documents, but also about understanding how different retrieval strategies behave under different query conditions. No single method was best for every situation. Future work could improve the system by adding BM25, testing more hybrid alpha values, using denser datasets, improving controlled query expansion, and adding reranking or user relevance feedback.

REFERENCES

- [1] [1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] [2] G. Salton, A. Wong, and C. S. Yang, “A vector space model for automatic indexing,” *Communications of the ACM*, vol. 18, no. 11, pp. 613–620, 1975.
- [3] [3] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [4] [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- [5] [5] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *Advances in Neural Information Processing Systems*, 2020.
- [6] [6] M. A. K. Ayoub, Z. Su, and Q. Li, “A case study of enhancing sparse retrieval using LLMs,” in *Companion Proceedings of the ACM Web Conference 2024*, pp. 1609–1615, 2024.
- [7] [7] R. Jagerman, H. Zhuang, Z. Qin, X. Wang, and M. Bendersky, “Query expansion by prompting large language models,” arXiv preprint arXiv:2305.03653, 2023.
- [8] [8] L. Wang, N. Yang, and F. Wei, “Query2doc: Query expansion with large language models,” arXiv preprint arXiv:2303.07678, 2023.
- [9] [9] K. Lang, “Newsweeder: Learning to filter netnews,” in *Proceedings of the Twelfth International Conference on Machine Learning*, pp. 331–339, 1995.
- [10] [10] Scikit-learn Developers, “The 20 Newsgroups text dataset,” *Scikit-learn Documentation*. Accessed for dataset loading and preprocessing support.
- [11] [11] Hugging Face, “Transformers: State-of-the-art machine learning for PyTorch, TensorFlow, and JAX.” Accessed for FLAN-T5 model implementation.
- [12] [12] Gradio Team, “Gradio: Build machine learning web apps.” Accessed for creating the interactive search and evaluation interface.